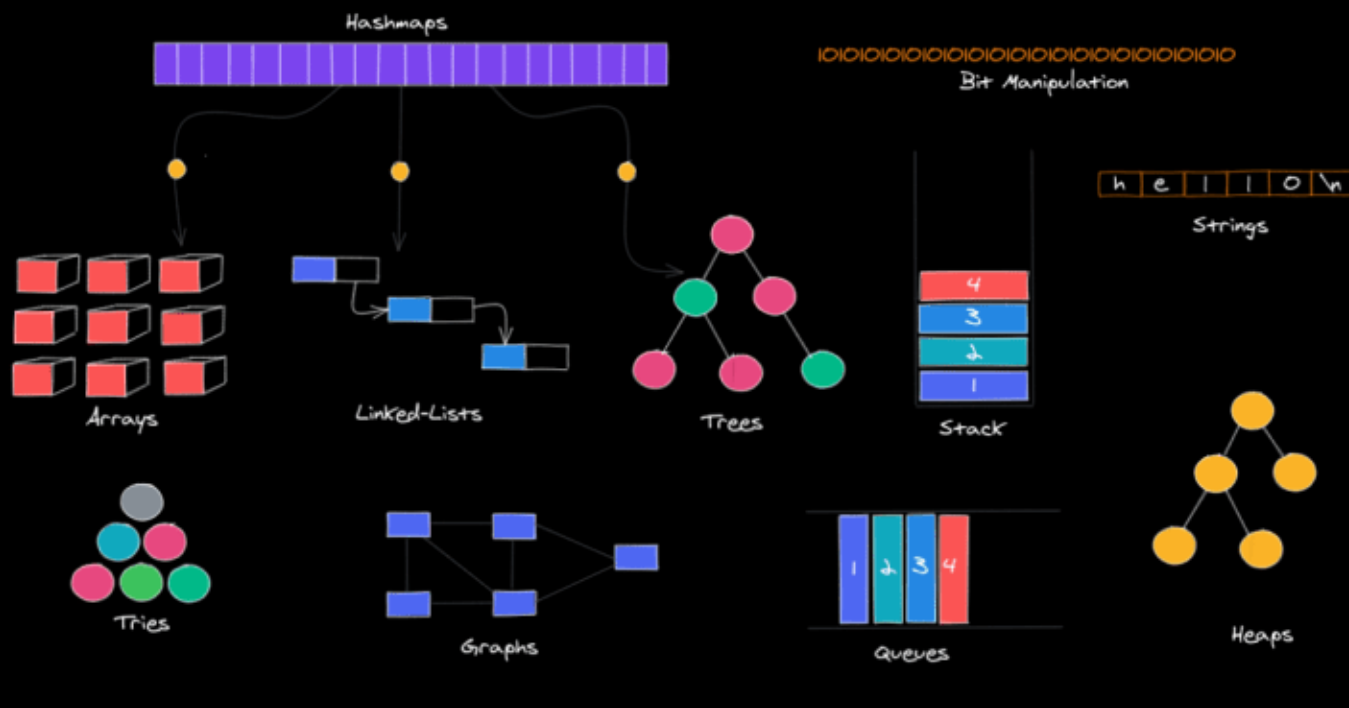




数据结构与算法导论

刘子博

FDSM

liuzibo@fudan.edu.cn

Today's Agenda

- 查找与映射
- 哈希

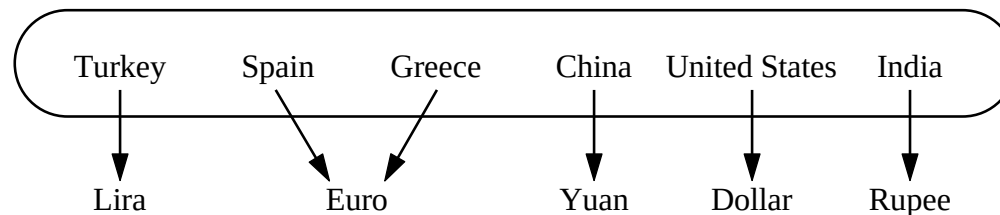
映射





映射

- 映射（map）是由键值对所构成的集合，其中的键是唯一的，但值不一定唯一
- 映射上的主要操作为查找、插入和删除
- 应用：
 - 地址簿
 - 学校里记录学生数据的数据库
 - 域名系统（DNS）



字典

- 字典 (dict) 类是Python中最重要的数据结构
 - dict类表示一种被称为字典的抽象概念，其中的每个键是唯一的，且被映射到对应的值上
 - dict类是一种无序映射的实现
- 为避免混淆，我们使用“字典”这一术语来指代Python的dict类，而使用“映射”这一术语来讨论更为一般的抽象数据类型概念

映射的ADT



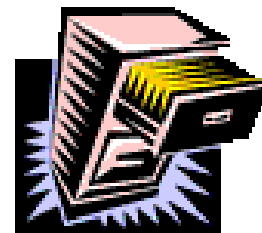
- 我们引入映射M的ADT，并定义其行为与dict类一致
- 映射M最重要的5类行为（查找、增添、删除、修改、遍历）：
 - $M[k]$: 如果存在，返回映射M中键k对应的值，否则抛出KeyError。Python中该功能由特殊方法__getitem__实现
 - $M[k] = v$: 使得映射M中键k对应于值v。如k有之前对应的值，则替换掉该值。Python中该功能由特殊方法__setitem__实现
 - $\text{del } M[k]$: 从映射M中删除键为k的键值对。如键k不存在则抛出KeyError。Python中该功能由特殊方法__delitem__实现
 - $\text{len}(M)$: 返回映射M中键值对的数量。Python中该功能由特殊方法__len__实现
 - $\text{iter}(M)$: 返回一个包含映射M所有键的迭代器。Python中该功能由特殊方法__iter__实现

映射的ADT



- 为实现其他方便的功能，映射M也应支持以下行为：
 - `k in M`: 如果映射M中包含键k则返回True，否则返回False。Python中该功能由特殊方法`__contains__`实现
 - `M.get(k, d=None)`: 如映射M中存在k则返回M[k]，否则返回默认值d。这提供了一种避免KeyError风险的M[k]查询方法
 - `M.setdefault(k, d)`: 如映射M中存在键k，则返回M[k]，否则令M[k] = d并返回d
 - `M.pop(k, d=None)`: 从映射M中删除键为k的键值对，并将其值v返回。如M中不存在键k则返回默认值d（如d为None则抛出KeyError）
 - `M.popitem()`: 从映射M中随机删除并返回一个键值对(k, v)。如M为空则抛出KeyError

映射的ADT



- 为实现其他方便的功能，映射M也应支持以下行为：
 - `M.clear()`: 删除映射M中所有的键值对
 - `M.keys()`: 返回M的所有键
 - `M.values()`: 返回M的所有值
 - `M.items()`: 返回M的所有键值对
 - `M.update(M2)`: 将M2中的所有键值对信息插入M中
 - `M == M2`: 如果映射M和M2中包含的所有键值对完全相同则返回True，否则返回False
 - `M != M2`: 如果映射M和M2中有不同的键值对则返回True，否则返回False

例

Operation	Return Value	Map
len(M)	0	{ }
M['K'] = 2	—	{'K': 2}
M['B'] = 4	—	{'K': 2, 'B': 4}
M['U'] = 2	—	{'K': 2, 'B': 4, 'U': 2}
M['V'] = 8	—	{'K': 2, 'B': 4, 'U': 2, 'V': 8}
M['K'] = 9	—	{'K': 9, 'B': 4, 'U': 2, 'V': 8}
M['B']	4	{'K': 9, 'B': 4, 'U': 2, 'V': 8}
M['X']	KeyError	{'K': 9, 'B': 4, 'U': 2, 'V': 8}
M.get('F')	None	{'K': 9, 'B': 4, 'U': 2, 'V': 8}
M.get('F', 5)	5	{'K': 9, 'B': 4, 'U': 2, 'V': 8}
M.get('K', 5)	9	{'K': 9, 'B': 4, 'U': 2, 'V': 8}
len(M)	4	{'K': 9, 'B': 4, 'U': 2, 'V': 8}
del M['V']	—	{'K': 9, 'B': 4, 'U': 2}
M.pop('K')	9	{'B': 4, 'U': 2}
M.keys()	'B', 'U'	{'B': 4, 'U': 2}
M.values()	4, 2	{'B': 4, 'U': 2}
M.items()	('B', 4), ('U', 2)	{'B': 4, 'U': 2}
M.setdefault('B', 1)	4	{'B': 4, 'U': 2}
M.setdefault('A', 1)	1	{'A': 1, 'B': 4, 'U': 2}
M.popitem()	('B', 4)	{'A': 1, 'U': 2}

映射的应用：单词频率统计

- 考虑统计一个文档中单词出现频率的问题
- 使用Python的dict类实现该映射：

```
freq = {}
for piece in open(filename).read().lower().split():
    # only consider alphabetic characters within this piece
    word = ''.join(c for c in piece if c.isalpha())
    if word:                                     # require at least one alphabetic character
        freq[word] = 1 + freq.get(word, 0)

max_word = ''
max_count = 0
for (w,c) in freq.items():    # (key, value) tuples represent (word, count)
    if c > max_count:
        max_word = w
        max_count = c

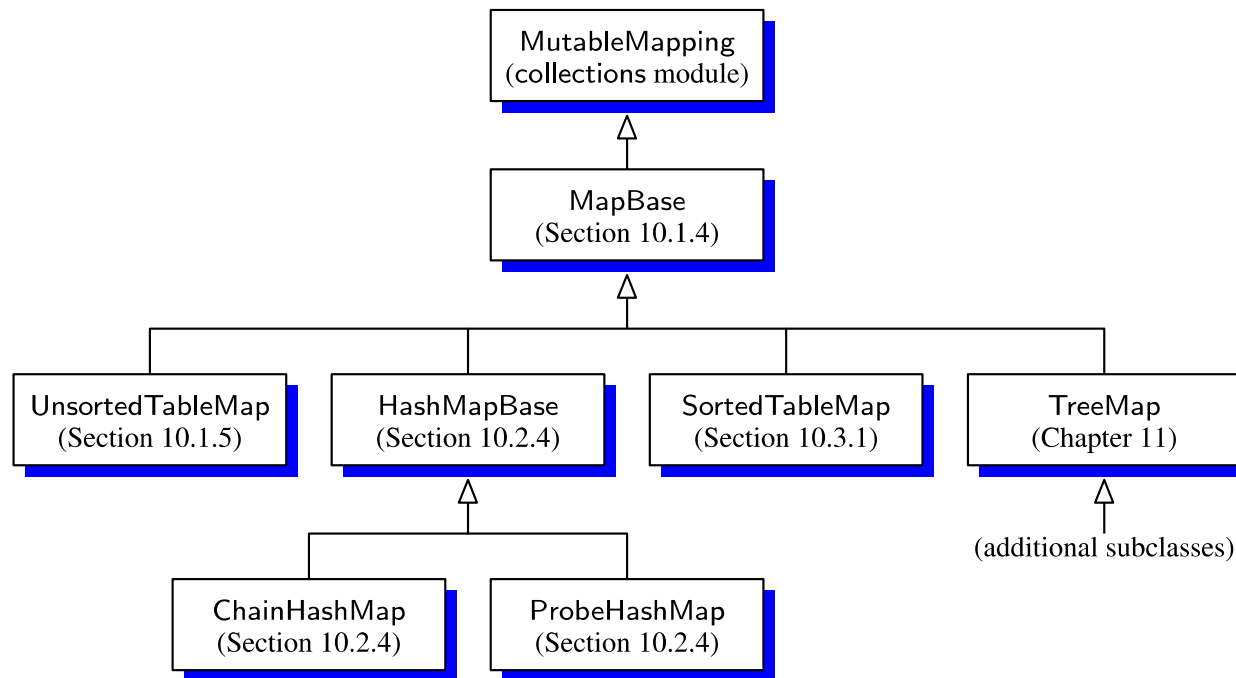
print('The most frequent word is', max_word)
print('Its number of occurrences is', max_count)
```

映射类

- 我们可以使用Python自带的抽象基类帮助我们构建自己的映射类
- Python的collections模块中有以下两个抽象基类：
 - Mapping: 包含dict类支持的所有不可变方法
 - MutableMapping: 包含dict类支持的不可变及可变方法
- MutableMapping类提供映射ADT中除以下5种行为外所有行为的具体实现: `__getitem__`, `__setitem__`, `__delitem__`, `__len__` 和 `__iter__`
- 我们只需在继承类中给出以上5种行为的具体实现即可得到映射ADT中所有行为的实现

映射类

- 为了实现更好的代码重用性，我们定义自己的MapBase类。此类继承自MutableMapping类，并为之后的各种映射类提供支持



MapBase类

```
class MapBase(MutableMapping):
    """Our own abstract base class that includes a nonpublic _Item class."""

    #----- nested _Item class -----
    class _Item:
        """Lightweight composite to store key-value pairs as map items."""
        __slots__ = '_key', '_value'

        def __init__(self, k, v):
            self._key = k
            self._value = v

        def __eq__(self, other):
            return self._key == other._key    # compare items based on their keys

        def __ne__(self, other):
            return not (self == other)        # opposite of __eq__

        def __lt__(self, other):
            return self._key < other._key     # compare items based on their keys
```

简单的无序映射类

```
class UnsortedTableMap(MapBase):
    """Map implementation using an unordered list."""

    def __init__(self):
        """Create an empty map."""
        self._table = []                                # list of _Item's

    def __getitem__(self, k):
        """Return value associated with key k (raise KeyError if not found)."""
        for item in self._table:
            if k == item._key:
                return item._value
        raise KeyError('Key Error: ' + repr(k))

    def __setitem__(self, k, v):
        """Assign value v to key k, overwriting existing value if present."""
        for item in self._table:
            if k == item._key:                            # Found a match:
                item._value = v                            # reassign value
                return                                      # and quit
        # did not find match for key
        self._table.append(self._Item(k,v))
```

简单的无序映射类

```
def __delitem__(self, k):
    """Remove item associated with key k (raise KeyError if not found)."""
    for j in range(len(self._table)):
        if k == self._table[j]._key:
            self._table.pop(j)
            return
    raise KeyError('Key Error: ' + repr(k))

def __len__(self):
    """Return number of items in the map."""
    return len(self._table)

def __iter__(self):
    """Generate iteration of the map's keys."""
    for item in self._table:
        yield item._key
```

Found a match:
remove item
and quit

yield the KEY

基于列表的映射类性能

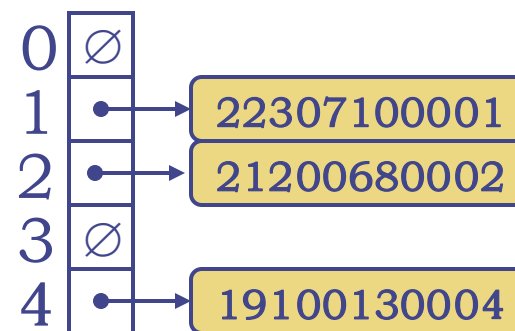
- 我们使用列表实现了一个无序映射类UnsortedTableMap
- UnsortedTableMap类的性能：
 - 插入操作只需在无序列表尾增加新元素，故运行时间为 $O(1)$
 - 查找及删除时，最坏情况下需要遍历整个列表，故运行时间为 $O(n)$
- 基于列表实现的无序映射只在映射规模较小或插入操作经常执行但查找和删除很少执行时较有效率
- 可否在更短时间内完成映射的**查找**功能？

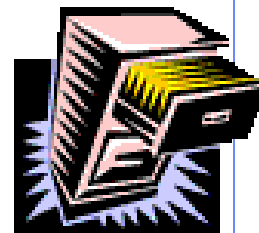


Today's Agenda

- 查找与映射
- 哈希

哈希表





哈希表

- 实践中最常用来实现映射的数据结构——**哈希表 (hash table)**
- 在哈希表中，我们**直接使用键作为索引**来查找键值对
- 考虑一种简单情况：映射M中含有n个元组，如使用0到N - 1的**整数**作为键 ($N \geq n$)，则可使用长为N的**查找表**来表示此映射：

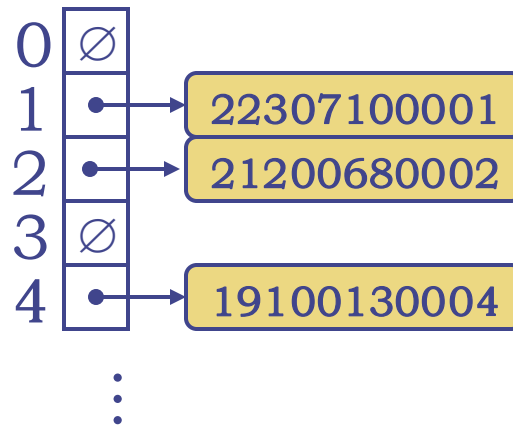
0	1	2	3	4	5	6	7	8	9	10
	D		Z			C	Q			

包含(1, D)、(3, Z)、(6, C)、(7, Q)且长度为11的哈希表

- 此时查找、插入、删除操作均可在 **$O(1)$** 时间内完成

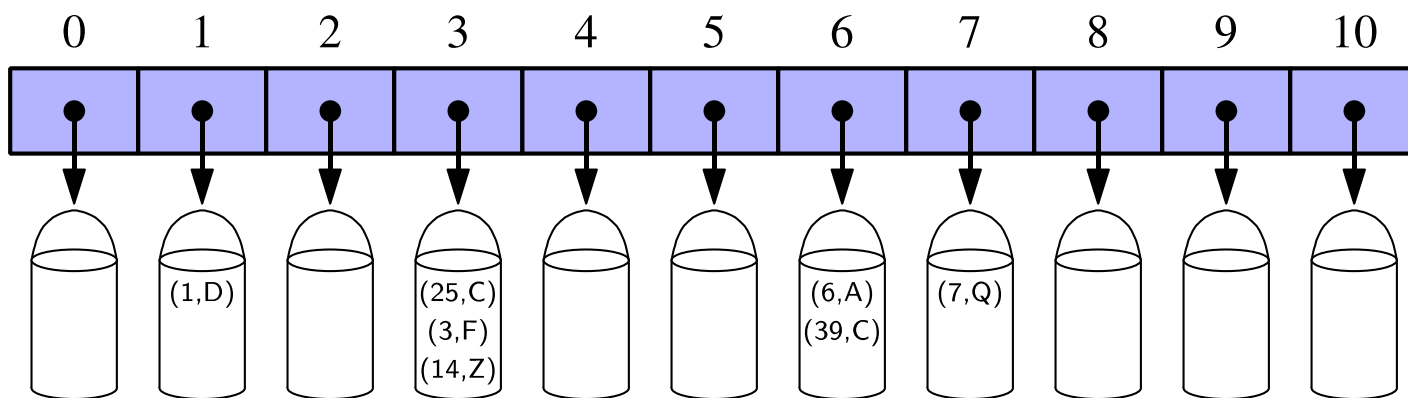
哈希表

- 将以上结构扩展到更一般情况：
 - 如果 $N \gg n$ ，如何减小分配的空间？
 - 映射的键不为整数的情况下，如何将其转换为索引？
- 使用**哈希函数** (hash function) 将键映射为表的索引
 - 例如：使用学号的后四位作为表的索引

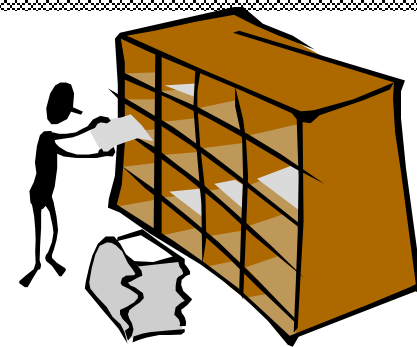


哈希表

- 理想情况下，哈希函数将键均匀地映射到 $0 \sim N - 1$ 这一范围内，但实践中可能有两个或更多不同键被映射到同一个索引上，这时需要进行**冲突 (collision) 处理**
- 我们将表概念化为**桶数组 (bucket array)**，每个桶里可能装着多个由哈希函数映射到该桶的键值对：



包含(1, D)、(25, C)、(3, F)、(14, Z)、(6, A)、
(39, C)、(7, Q)且长度为11的桶数组



哈希表与哈希函数

- 哈希函数 h 将键映射到区间 $[0, N - 1]$ 内的整数
- 例：

$$h(x) = x \bmod N$$

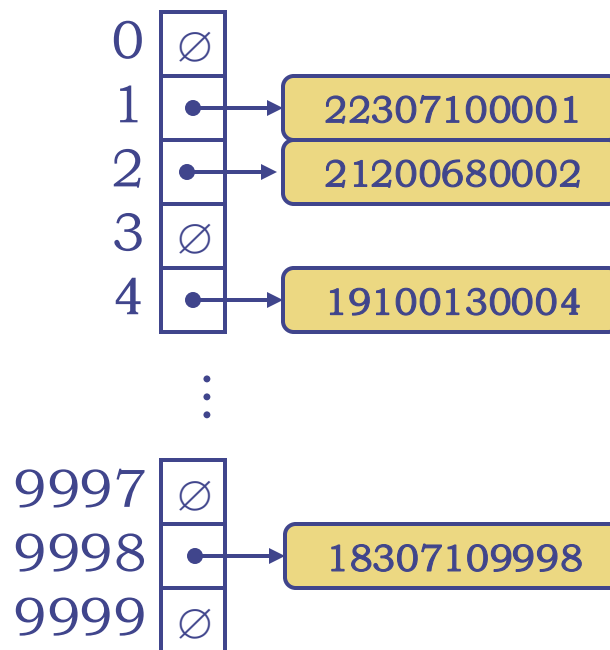
此哈希函数可对任意整数键 x 使用

- 整数 $h(x)$ 被称为键 x 的哈希值 (hash value)
- 对于给定类型的键，哈希表由以下两部分构成：
 - 哈希函数 h
 - 长为 N 的数组（表）
- 借助哈希表实现映射时，我们的目标是在索引 $i = h(k)$ 处存储键值对 (k, v)

例：存储学生信息

- 我们使用哈希表来存储诸如（学号，姓名）的条目，其中学号为11位正整数
- 哈希表的数组长度为 $N = 10,000$ ；
哈希函数为：

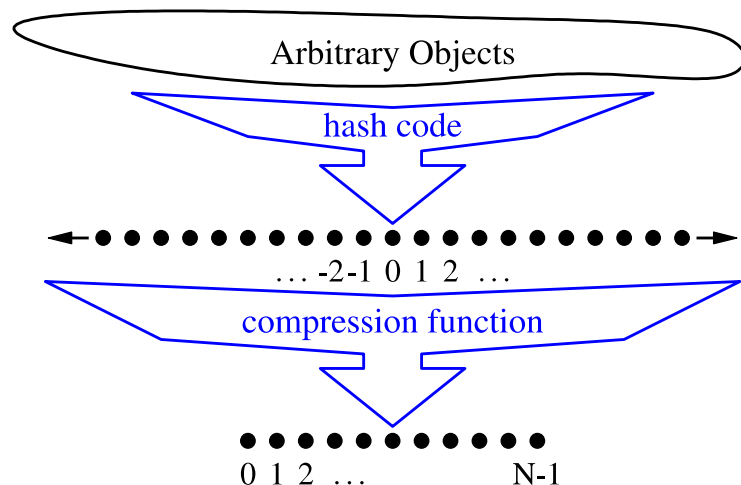
$$h(x) = x \text{ 的后四位}$$





哈希函数

- 哈希函数通常可被视为由两部分组成：
- 哈希码 (hash code) :
 $h_1: \text{键} \rightarrow \text{整数}$
- 压缩函数 (compression function) :
 $h_2: \text{整数} \rightarrow [0, N-1]$

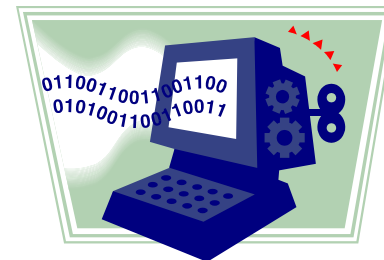


- 哈希码先作用于键上，压缩函数再作用于哈希码所得的结果，即：

$$h(x) = h_2(h_1(x))$$

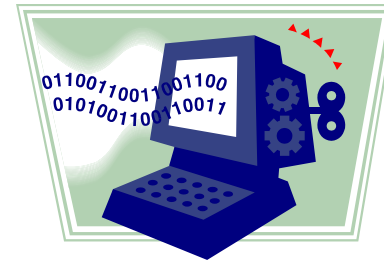
- 哈希函数的目标是以随机的方式将键“分散”开，因此哈希也被称为**散列**

哈希码



- 地址法：
 - 直接使用键的地址作为哈希码
- 整数转换法：
 - 将键的位表示作为整数处理
 - 适用于键的位数不大于整数位数的情况

哈希码



- 分量求和法：
 - 将键的位表示分为固定长度的几部分（如16或32位），再将各部分相加，忽略溢出
- 分量异或法：
 - 将键的位表示分为固定长度的几部分（如16或32位），再将各部分进行位异或运算
- 适用于键的位数大于整数位数的情况
- 可能存在的问题：忽略了各部分间的顺序，对字符串或可变长度对象不一定是个好选择
 - temp01/temp10, stop/tops/pots/spot

哈希码

- 多项式哈希码:

- 先将键的位表示分为固定长度的几部分（如16或32位）：

$$x_0 x_1 \dots x_{n-1}$$

- 选择一个非零常数 a ，计算如下多项式的值，忽略溢出：

$$p(a) = x_0 a^{n-1} + x_1 a^{n-2} + \dots + x_{n-2} a + x_{n-1}$$

- 处理英文字符串时，33、37、39、41特别适合选作 a 值，在超过50000个英文单词构成的集合上每个哈希码上的冲突小于7次

哈希码

- 利用Horner规则，多项式 $p(a)$ 的值可以在 $O(n)$ 内计算出来：
 - 根据以下递推公式，已知前一个多项式的值时，每个多项式的值均可在 $O(1)$ 的时间内计算出来：

$$p_0(a) = x_0$$

$$p_i(a) = x_i + ap_{i-1}(a) \quad (i = 1, 2, \dots, n-1)$$

- 我们想得到的 $p(a) = p_{n-1}(a)$
- 即以如下方式计算多项式：

$$p(z) = x_{n-1} + a(x_{n-2} + a(x_{n-3} + \dots + a(x_2 + a(x_1 + ax_0))\dots))$$

哈希码

- 循环移位哈希码：
 - 多项式哈希码的一个变种，将多项式哈希码中乘 a 的操作变为对部分和的循环移位
 - 如对以下32位数进行5位循环移位：

00111101100101101010100010101000

- 则可得到：

10110010110101010001010100000111

- 5位循环移位在230000个单词构成的集合中哈希码的最大冲突次数为3

哈希码

- Python中的哈希码：
 - Python中使用内置函数hash(x)计算对象x的哈希码
 - 只有不可变数据类型才是可哈希的
 - ◆ int、float、str、tuple、frozenset可哈希
 - ◆ list、dict、set不可哈希
 - 默认情况下，用户定义的类不可哈希。但可通过在类中实现一个使用不可变对象计算哈希码的特殊方法__hash__，有此方法的类即可哈希，如：

```
def __hash__(self):  
    return hash((self._red, self._green, self._blue))    # hash combined tuple
```

压缩函数



- 模函数（取余）：

$$h_2(y) = y \bmod N$$

- 哈希表的大小 N 通常取质数
- 如 N 不为质数，则出现冲突的概率大幅提升
 - ♦ 例：哈希码为{200, 205, 210, 215, 220, ..., 600}。如 $N = 100$ ，则每个哈希码将至少与另三个哈希码冲突；但如 $N = 101$ ，则不会发生冲突



压缩函数

- MAD方法:

$$h_2(y) = [(ay + b) \bmod p] \bmod N$$

- 即Multiply-Add-and-Divide
- p 为比 N 大的质数, a 和 b 为区间内任意选择的整数, 且 $a > 0$
- 该函数使得哈希函数得到的结果更好地被分散在 $[0, N - 1]$ 中

哈希表的Python实现

```
class HashMapBase(MapBase):
    """Abstract base class for map using hash-table with MAD compression.

    Keys must be hashable and non-None.
    """
    def __init__(self, cap=11, p=109345121):
        """Create an empty hash-table map.

        cap        initial table size (default 11)
        p          positive prime used for MAD (default 109345121)
        """
        self._table = cap * [ None ]
        self._n = 0                # number of entries in the map
        self._prime = p            # prime for MAD compression
        self._scale = 1 + randrange(p-1) # scale from 1 to p-1 for MAD
        self._shift = randrange(p)    # shift from 0 to p-1 for MAD

    def _hash_function(self, k):
        return (hash(k)*self._scale + self._shift) % self._prime % len(self._table)

    def __len__(self):
        return self._n
```

哈希表的Python实现

```
def __getitem__(self, k):
    j = self._hash_function(k)
    return self._bucket_getitem(j, k)           # may raise KeyError

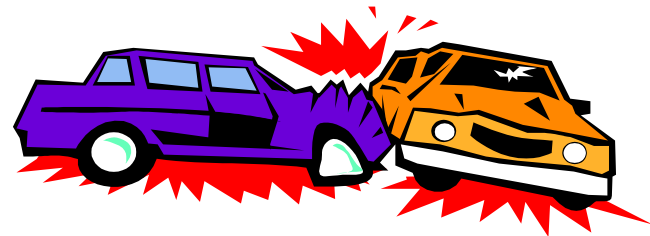
def __setitem__(self, k, v):
    j = self._hash_function(k)
    self._bucket_setitem(j, k, v)              # subroutine maintains self._n
    if self._n > len(self._table) // 2:        # keep load factor <= 0.5
        self._resize(2 * len(self._table) - 1) # Use number 2*x - 1 as new size

def __delitem__(self, k):
    j = self._hash_function(k)
    self._bucket_delitem(j, k)                 # may raise KeyError
    self._n -= 1

def _resize(self, c):
    """Resize bucket array to capacity c and rehash all items."""
    old = list(self.items())                   # use iteration to record existing items
    self._table = c * [None]                  # then reset table to desired capacity
    self._n = 0                               # n recomputed during subsequent adds
    for (k,v) in old:
        self[k] = v                           # reinsert old key-value pair
```



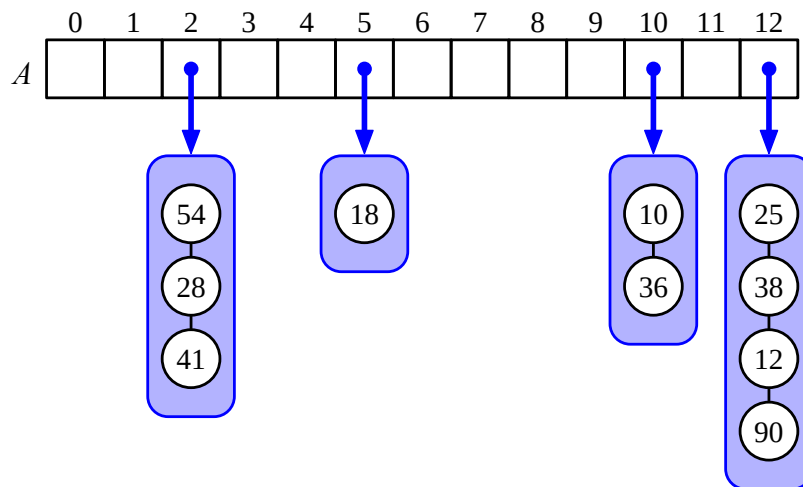
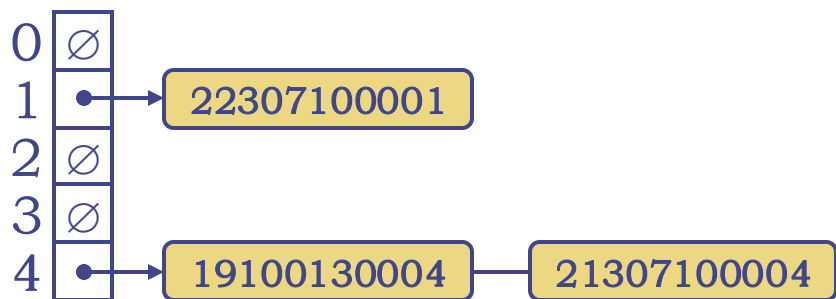
冲突处理



- 当两个不同的键被映射到同一个索引时，就发生了冲突
- 哈希函数的设计可以减少冲突，但往往不能完全避免冲突的发生
- 冲突处理的方法选取会直接影响性能
- 我们有两类常见的冲突处理方式：链地址法（separate chaining）、开放寻址法（open addressing）

链地址法

- 链地址法 (sepearate chaining) : 在桶数组中的每个位置建立一个二级容器, 存储所有映射到该位置的键值对; 此容器通常为数组或链表
- 链地址法实现简单, 但需要额外的空间



链地址法

- ### 链地址法的基本操作:

```

Algorithm get(k):
    return A[h(k)].get(k)

```

```

Algorithm set(k,v):
    t = A[h(k)].set(k,v)
    if t = null then                                {k is a new key}
        n = n + 1
    return t

```

```

Algorithm remove(k):
    t = A[h(k)].remove(k)
    if t  $\neq$  null then                                {k was found}
        n = n - 1
    return t

```

链地址法的Python实现

```
class ChainHashMap(HashMapBase):
    """Hash map implemented with separate chaining for collision resolution."""

    def _bucket_getitem(self, j, k):
        bucket = self._table[j]
        if bucket is None:
            raise KeyError('Key Error: ' + repr(k))           # no match found
        return bucket[k]                                       # may raise KeyError

    def _bucket_setitem(self, j, k, v):
        if self._table[j] is None:
            self._table[j] = UnsortedTableMap()               # bucket is new to the table
        oldsize = len(self._table[j])
        self._table[j][k] = v
        if len(self._table[j]) > oldsize:                       # key was new to the table
            self._n += 1                                         # increase overall map size
```


链地址法的Python实现

```
def _bucket_delitem(self, j, k):  
    bucket = self._table[j]  
    if bucket is None:  
        raise KeyError('Key Error: ' + repr(k))  
    del bucket[k]  
  
    # no match found  
    # may raise KeyError  
  
def __iter__(self):  
    for bucket in self._table:  
        if bucket is not None:  
            for key in bucket:  
                yield key  
  
    # a nonempty slot
```

链地址法的性能分析

- 对需要存放 n 个元素且长为 N 的哈希表，定义其负载因子 (load factor) 为 $\lambda = n/N$
- 如 λ 为 $O(1)$ ，则哈希表的核心操作时间复杂度也为 $O(1)$
- λ 应选择较小的常数，最好不大于1
- 对于链地址法，冲突发生后找到目标键的时间与链的长度成正比； λ 接近1时，冲突发生的概率急剧上升
- 一般来说，建议将链地址法的负载因子控制在 $\lambda < 0.9$

开放寻址法

- 链地址法需要占用额外空间存储链，在空间宝贵时可能成为问题
- 开放寻址法 (open addressing) 直接将键值对存储在对应索引处，并采取更复杂的机制处理冲突
- 开放寻址法需要哈希表长度 N 始终大于所需存储的键值对数目 n
- 冲突发生时，当前键值对的位置已被另一个键值对占据，则根据某种规则探测下一个位置尝试进行插入
 - 线性探测法、二次探测法、双重哈希法……

线性探测法

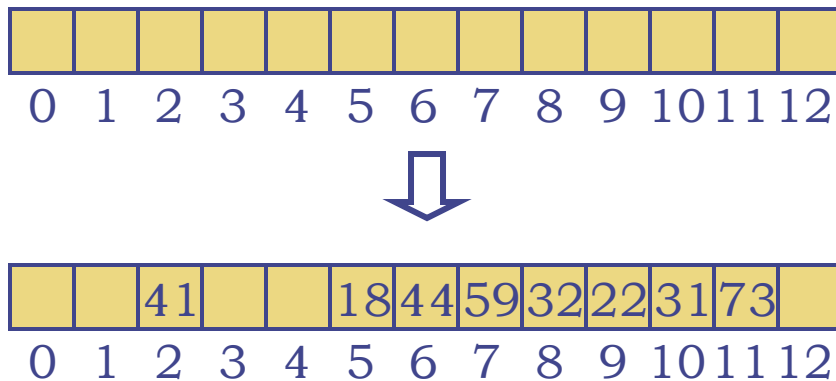
- 线性探测法 (linear probing) : 冲突发生时, 尝试将新键值对插入紧邻的下一个位置
 - 如 $j = h(k)$, 且索引 j 处已被占据, 则尝试将新键值对存 $(j + 1) \bmod N$ 处; 如还被占据, 则尝试 $(j + 2) \bmod N \dots\dots$
- 每次查看某位置是否可存储数据的操作被称为一次“探测”
- 可能存在的问题: 倾向于将键值对集中连续地存储, 很大程度上影响搜索效率

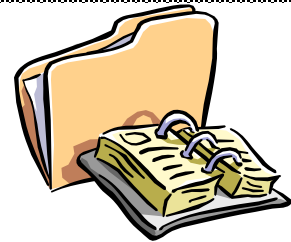
线性探测法

· 例:

$$h(x) = x \bmod 13$$

· 以如下顺序插入键: 18、41、22、44、59、32、31、73





线性探测法

- 我们在哈希表 A 上考虑线性探测法的查找操作 $\text{get}(k)$:
 - 从索引 $h(k)$ 处开始探测
 - 连续探测下一个位置，直到发生以下某种情况
 - 找到一个键为 k 的键值对
 - 找到一个未存储键值对的空位置
 - 遍历 N 个位置后未出现以上两种情况

Algorithm $\text{get}(k)$

```
 $i \leftarrow h(k)$   
 $p \leftarrow 0$   
repeat  
   $c \leftarrow A[i]$   
  if  $c = \emptyset$   
    return null  
  else if  $c.\text{getKey}() = k$   
    return  $c.\text{getValue}()$   
  else  
     $i \leftarrow (i + 1) \bmod N$   
     $p \leftarrow p + 1$   
until  $p = N$   
return null
```

线性探测法

- 对于插入和删除操作，由于直接删除元素可能会导致查找其他元素失败，我们需要引入一个新对象 *AVAIL* 表示已删除的元素
- 删除操作 **remove(k)**:
 - 使用查找操作 **get(k)** 寻找键为 k 的元素
 - 如果找到这样的元素 (k, v) ，则将其替换为 *AVAIL*，并返回 v
 - 否则，返回 *null*

		41			18	44	59	32	22	31	73	
0	1	2	3	4	5	6	7	8	9	10	11	12



直接删除 59 会使得对 32 的查找失败

		41			18	44		32	22	31	73	
0	1	2	3	4	5	6	7	8	9	10	11	12

线性探测法

- 插入操作 $\text{set}(k)$:
 - 从索引 $h(k)$ 处开始探测
 - 连续探测下一个位置，直到发生以下情况：
 - ♦ 找到一个空位置或存储着 AVAIL 的位置
 - ♦ 遍历 N 个位置后未出现以上情况，插入不成功
 - 如成功找到插入位置，则将键值对 (k, v) 存在此处

线性探测法的Python实现

```
class ProbeHashMap(HashMapBase):
    """Hash map implemented with linear probing for collision resolution."""
    _AVAIL = object()          # sentinel marks locations of previous deletions

    def _is_available(self, j):
        """Return True if index j is available in table."""
        return self._table[j] is None or self._table[j] is ProbeHashMap._AVAIL

    def _find_slot(self, j, k):
        """Search for key k in bucket at index j.

        Return (success, index) tuple, described as follows:
        If match was found, success is True and index denotes its location.
        If no match found, success is False and index denotes first available slot.
        """
        firstAvail = None
        while True:
            if self._is_available(j):
                if firstAvail is None:
                    firstAvail = j                # mark this as first avail
                if self._table[j] is None:
                    return (False, firstAvail)    # search has failed
                elif k == self._table[j]._key:
                    return (True, j)              # found a match
                j = (j + 1) % len(self._table)    # keep looking (cyclically)
```

线性探测法的Python实现

```
def _bucket_getitem(self, j, k):
    found, s = self._find_slot(j, k)
    if not found:
        raise KeyError('Key Error: ' + repr(k))           # no match found
    return self._table[s]._value

def _bucket_setitem(self, j, k, v):
    found, s = self._find_slot(j, k)
    if not found:
        self._table[s] = self._Item(k,v)                  # insert new item
        self._n += 1                                       # size has increased
    else:
        self._table[s]._value = v                          # overwrite existing

def _bucket_delitem(self, j, k):
    found, s = self._find_slot(j, k)
    if not found:
        raise KeyError('Key Error: ' + repr(k))           # no match found
    self._table[s] = ProbeHashMap._AVAIL                  # mark as vacated

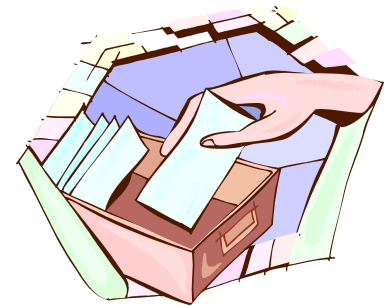
def __iter__(self):
    for j in range(len(self._table)):                      # scan entire table
        if not self._is_available(j):
            yield self._table[j]._key
```

二次探测法

- 二次探测法 (quadratic probing) : 冲突发生时, 尝试依次探测以下位置, 直到发现一个空位置:

$$A[(h(k) + f(i)) \bmod N], i = 0, 1, 2, \dots$$

- 其中 $f(i) = i^2$
- 二次探测法创造了其独有的聚集方式, 即二次聚集
- 当 N 为质数, 且负载因子小于0.5时, 二次探测能保证找到一个空位置



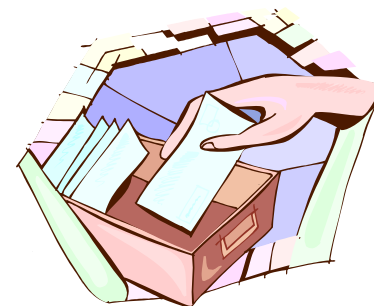
双重哈希法

- 双重哈希法 (double hashing) : 使用第二个哈希函数 $h'(k)$ 确定下一个探测的位置。具体来说, 冲突发生时, 依次探索以下位置, 直到发现一个空位置:

$$A[(h(k) + f(i)) \bmod N], i = 0, 1, 2, \dots$$

- 其中 $f(i) = ih'(k)$
- 第二个哈希函数 $h'(k)$ 应保证不为0
- 哈希表的长度 N 应保证为质数, 以使得所有位置都可被探测到

双重哈希法



- 常用的第二个哈希函数 $h'(k)$ 为：

$$h'(k) = q - (k \bmod q)$$

- 其中 $q < N$ 且为质数
- 这种情况下，第二个哈希函数 $h'(k)$ 的取值范围为：

$$1, 2, \dots, q$$

双重哈希法

- 考虑一个采取双重哈希法处理冲突的哈希表如下：

- $N = 13$

- $h(k) = k \bmod 13$

- $h'(k) = 7 - (k \bmod 7)$

- 将键18、41、22、44、59、32、31、73依次插入

k	$h(k)$	$h'(k)$	Probes	
18	5	3	5	
41	2	1	2	
22	9	6	9	
44	5	5	5	10
59	7	4	7	
32	6	3	6	
31	5	4	5	9 0
73	8	4	8	

0	1	2	3	4	5	6	7	8	9	10	11	12



31		41			18	32	59	73	22	44		
0	1	2	3	4	5	6	7	8	9	10	11	12

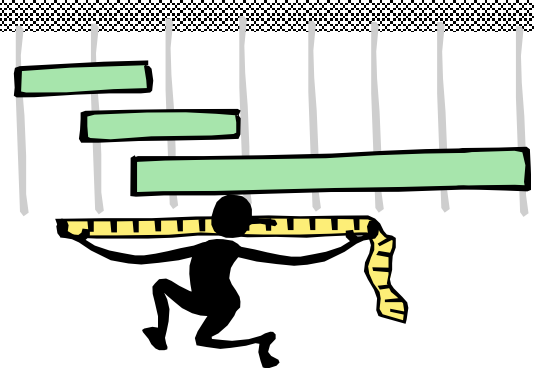
开放寻址法的性能分析

- 随着负载因子 λ 增长到0.5以上并向1逼近时，开放寻址法下桶数组中元素的集群开始增加，这使得各探测策略均需花费较长时间寻找空位置
- 实验表明，对于线性探测法，应保持 $\lambda < 0.5$ ，对其他探测法负载因子可略高（Python实现的开放寻址法规定 $\lambda < 2/3$ ）

重新哈希

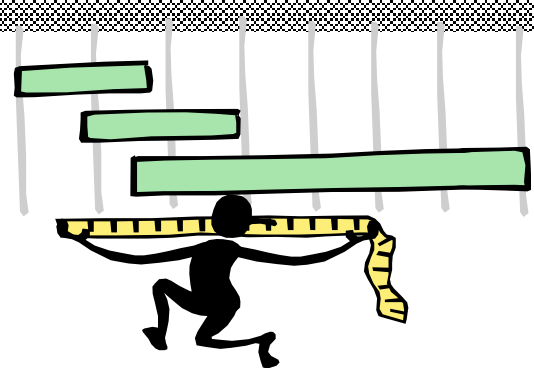
- 当对哈希表的插入操作导致负载因子超过给定阈值时，需要调整哈希表的大小并将所有对象重新插入新的哈希表，这被称为**重新哈希 (rehashing)**
- 不需要重新定义**哈希码**，但需要基于新哈希表重新设计一个**压缩函数**
- 新哈希表大小通常约为原表大小的2倍
- 建立新哈希表的时间被摊销到所有插入操作上

哈希表的效率



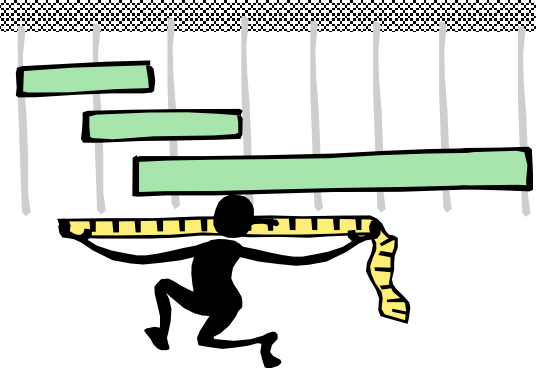
- 最坏情况下，哈希表的查找、插入和删除操作均需花费 $O(n)$ 的时间
- 负载因子 λ 对哈希表的效率有很大的影响
- 假设哈希值的分布是随机的，则在开放寻址法下，期望的探测次数为 $\frac{1}{1-\lambda}$

哈希表的效率



- 平均来说，哈希表的查找、插入和删除操作仅需花费 $O(1)$ 的时间
- 在实践中，当负载因子 λ 显著小于1时，哈希表的速度非常快
- 哈希表的应用：
 - 小型数据库
 - 编译器
 - 浏览器缓存

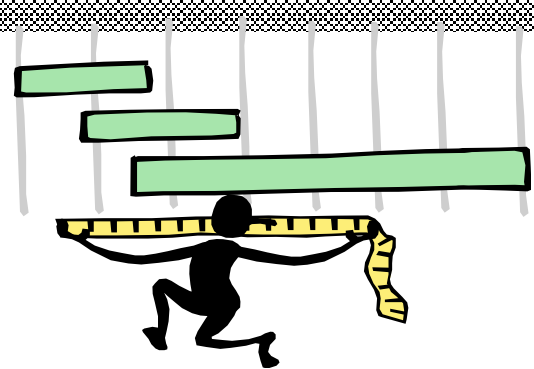
哈希表的效率



- 哈希表与无序列表的运行时间对比：

Operation	List	Hash Table	
		expected	worst case
--getitem--	$O(n)$	$O(1)$	$O(n)$
--setitem--	$O(n)$	$O(1)$	$O(n)$
--delitem--	$O(n)$	$O(1)$	$O(n)$
--len--	$O(1)$	$O(1)$	$O(1)$
--iter--	$O(n)$	$O(n)$	$O(n)$

哈希的特点



- 实现简单，可提供 $O(1)$ 时间内的查找性能
- 合适的哈希函数有时不易寻找
- 最坏情况下，查找性能恶化
- 忽略数据本身的逻辑联系，不利于排序等操作，只适合实现无序映射



Enjoy